

Using Objects

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Poll: What gets printed?

```
class Cat:
    def __init__(self, name: str, human: str):
        self.name = name
        self.human = human
        print(name * 3)
```

```
mini: Cat = Cat('Mini', 'Rasika')
```

1. Mini
2. <__main__.Cat object at 0x10d380200>
3. MiniMiniMini
4. (Nothing)

State and aliasing

When a variable holds an object, it holds a *reference* to that object. That object is stored somewhere in the memory of the computer, and the variable knows where to access it.

Multiple variables can hold references to the same object.

Alias: a second reference to an existing object (same place in computer's memory)

Copy: another object that looks exactly the same (different place in computer's memory)

```
original: list[int] = [1, 2, 3]

alias: list[int] = original
copy: list[int] = original.copy()

original[2] = 90

print(alias) # [1, 2, 90]
print(copy) # [1, 2, 3]
```

Passing mutable objects as arguments

If we pass an object as an argument to a function, an *alias* is created, not a copy.

```
def sum_with_bad_manners(in_list: list[int]) -> int:
    sum: int = 0
    while (len(in_list) > 0):
        sum += in_list.pop()
    return sum

my_list: list[int] = [1, 2, 3, 4]
print(f'Sum: {sum_with_bad_manners(my_list)}') # Sum: 10
print(my_list) # []
```

Good manners: Functions should leave their args unchanged (unless they clearly state otherwise in the documentation).

```
class Shirt:
    def __init__(self, size: int):
        self.size = size

def main() -> None:
    six_hundred: int = 600
    subtract_one(six_hundred)
    print(six_hundred)                # 600

    shirt: Shirt = Shirt(600)
    shrink_shirt(shirt)
    print(shirt.size)                 # 599

def subtract_one(num: int) -> None:
    num -= 1

def shrink_shirt(shirt: Shirt) -> None:
    shirt.size -= 1
```

Poll: Which of these makes it print 4?

```
def main() -> None:
    shirt: Shirt = Shirt(600)
    modify_shirt(shirt)
    print(shirt.size)

if __name__ == '__main__':
    main()
```

1.

```
def modify_shirt(shirt: Shirt) -> None:
    shirt.size = 4
```

2.

```
def modify_shirt(shirt: Shirt) -> None:
    shirt = Shirt(4)
```

3.

```
def modify_shirt(shirt: Shirt) -> Shirt:
    return Shirt(4)
```

Poll: Why does this test fail?

```
class Rectangle:

    def __init__(self, length: int, width: int) -> None:
        self.length = length
        self.width = width

class TestRectangle:
    def test_length_width(self) -> None:
        rect1 = Rectangle(3, 4)
        rect2 = Rectangle(4, 3)
        assert rect1 == rect2
```

1. Because you can't put tests inside a class like that.
2. Because the length and width are switched in rect1 and rect2. If they were both (3, 4), then it would pass.
3. Because `==` calls the `eq()` method, and `Rectangle` doesn't have an

Before overwriting `__eq__()` :

```
class Student():
    def __init__(self,
                  student_id: str, major: str
    ):
        self.id = student_id
        self.major = major
        self.courses: set[str] = set()

s1 = Student('s1', 'CS')
s2 = Student('s1', 'CS')
print(s1 == s2)  # False
```

After overwriting `__eq__()` :

```
class Student():
    def __init__(self,
                  student_id: str, major: str
    ):
        self.id = student_id
        self._major = major

    def __eq__(
        self, other: object
    ) -> bool:
        if not isinstance(other, Student):
            return False
        return self.id == other.id

s1 = Student('s1', 'CS')
s2 = Student('s1', 'CS')
print(s1 == s2)  # True
```

Poll: Why is this bad?

```
class Cat:
    def __init__(self, name: str):
        self.name = name
        self.food: list[str] = ['tuna', 'chicken']

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Cat):
            return False
        return other.name in self.food
```

1. It's possible for `cat_a` to equal `cat_b` today, but for `cat_a` to not be equal to `cat_b` tomorrow (with no code changes)
2. It's possible for `cat_a` to not equal itself
3. It's possible for `cat_a` to equal `cat_b`, and `cat_b` to not equal `cat_a`
4. All cats will be equal, making the `__eq__()` function useless

The `__str__()` function

Every class has a `__str__()` method.

We can overwrite it with our own `__str__()` method.

When we print an object, it implicitly calls the `__str__()` method.

The default `__str__()` method is not helpful.

```
mini = Cat('Mini', 'Rasika')  
print(mini) # <__main__.Cat object at 0x1095ca790
```

The `__str__()` function

Why is it different when we print a list (which is also an object)?

We usually write our own `__str__()` method that returns a more helpful `str` for that class.

```
class Cat:
    ...
    def __str__(self):
        return f'{self.name} meows to {self.human}'

mini = Cat('Mini', 'Rasika')
print(mini) # Mini meows to Rasika
```

Poll: What is printed?

```
class Cat:
    def __init__(self, name: str, human: str):
        self.name = name
        self.human = human

    def __str__(self) -> str:
        print('MUAHAHAHAHHA')
        return self.name

mini: Cat = Cat('Mini', 'Rasika')
print(mini)
```

1. Mini
2. MUAHAHAHAHHA // Mini
3. <__main__.Cat object at 0x10d380200>
4. MUAHAHAHAHHA // <__main__.Cat object at 0x10d380200>

The Single Responsibility Principle

A core principle for writing code is that every component of our code must have a single purpose. This makes our code easier to read, test, and maintain.

Component	Does not Follow Single Responsibility Principle
Variable	<code>age_or_nonexistent: int = -1 # negative if nonexistent</code>
Function	<code>def read_file_compute_average_print_score(filename: str) -> None:</code>
Class	<code>class FileManagerAndOutputFormatter</code>

Abstraction

We organize lines of code into functions, functions into classes, classes into...

"Abstraction": the procedure of grouping more granular things into less granular groups

Benefits of abstraction we already saw in functions:

- reuse code without redundancy
- hide implementation details
- break down a problem into smaller, more manageable pieces

The same applies to the idea of putting methods into classes.

Abstraction

Hard to debug or modify:

```
length: int = 5
width: int = 3

print(get_area_of_rectangle(
    length, width))

length = 6
width = 4

print(get_perimeter_of_rectangle(
    length, width))
```

Put relevant perimeter and area methods into a class for each shape:

```
table: Rectangle = Rectangle(5, 3)
print(table.area())

print(Rectangle(6, 4).perimeter())

chair: Square = Square(4)
print(chair.area())
```

Benefits of abstraction:

- We can use the same code multiple times without re-writing it
- It's easier to read
- It's easier to maintain and adapt the code later on
- It's easier to test behaviors in isolation
- Each variable, function, and class has a single, clear responsibility

(If time) Generic types

In Python:

We are able to put objects of different types into the same `list`, but it's discouraged (makes the list harder to process).

In CS2100:

Elements of a list must be of the same type since we require types in our Python code.

What would be the type of the variable `my_list = [1, 'a']` ?

The same `list` class can make objects of different types (`list[str]` and `list[int]`) because `list` is a *generic* type.

Define our own generic type:

1. First define the type variable `T`
2. Then use `T` to define the generic type `Stack[T]`
3. Inside the class `Stack[T]`, the `T` can be any type, but all instances of `T` must be the same type as each other
4. Instantiate it as `my_stack`, with `T` taking the value `int`
5. Instantiate another variable `my_other_stack`, where `T` is `str`

```
from typing import TypeVar, Generic

T = TypeVar('T')

class Stack(Generic[T]):
    def __init__(self) -> None:
        self.items: list[T] = []

    def push(self, item: T) -> None:
        self.items.append(item)

    def pop(self) -> T:
        return self.items.pop()

    def is_empty(self) -> bool:
        return not self.items

my_stack: Stack[int] = Stack()
my_stack.push(4)
print(my_stack.pop())    # 4
my_other_stack: Stack[str] = Stack()
```

Definitions

- **Generic type:** a class with a type variable, like `list[T]`
- **Parameterized type:** a generic type with the type variables filled in, like `list[str]`
- **Raw type:** a generic type without the type variable, like `list`
 - use this if we don't need to re-use the type variable anywhere else in the code

We can parametrize the type using another user-defined type:

```
stack_of_stacks: Stack[Stack[int]] = Stack()
```

Poll: Which of these is allowed?

```
class Thing(Generic[T]):  
    def __init__(self, item: Optional[T]):  
        """Item is of type T or None"""  
        self.item = item
```

1. `item: Thing[str] = Thing('hello')`
2. `item: Thing[str] = Thing(None)`
3. `item: Thing[str] = Thing(5)`
4. `item: Thing[Thing[str]] = Thing(Thing('hello'))`

Poll:

- 1. What is your main takeaway from today?**
- 2. What would you like to revisit next time?**